Laboratory Work 4: Advanced SQL Queries
Airport Database

DATABASE STRUCTURE

airline (6 rows)
Airline codes, names, countries and timestamps.

airport (5 rows)
Airport names and locations; referenced twice by flights.

passengers (5 rows)
Passenger identities, birth dates and passport details.

flights (6 rows)
Scheduled and actual times; references airline and airports.

booking (6 rows)
Platforms and prices; references flights and passengers.

baggage (6 rows)
Baggage weights; references booking.

baggage_check (6 rows)
Inspection results and timestamps; references booking and passengers.

EXECUTION AND INTERPRETATION

Run 01_schema.sql, then 02_sample_data.sql in a dedicated practice database. The schema script drops the seven tables with CASCADE. Run each SELECT in 03_lab4_queries.sql separately.

All tables have integer primary keys and NOT NULL columns. Foreign keys enforce the relationships above, including booking.flight_id -> flights.flight_id.

The following images show the exact queries and verified results from PostgreSQL 18.3. They are generated result illustrations, not screenshots of pgAdmin 4. The original assignment requests full-window pgAdmin screenshots; these illustrations do not document that interface.

Task 3 uses airline_id IN (1, 2), meaning either airline. A single flight cannot belong to both airlines in this schema. flight_id is used as the flight number.

Reginal is kept exactly as written. Earlier updated_at values are intentional test anomalies. Task 15 compares both month and year. The seven-table scope and minimal dataset follow the current request; earlier Lab 3 changes are not replayed.

Task 1

Retrieve all airline names in uppercase.

EXPLANATION

UPPER converts the displayed name to uppercase. It does not change the stored data.

VERIFIED RESULT

6 rows, including AIR FRANCE and KAZAIR.

TASK 01 | SQL AND VERIFIED RESULT

```
SELECT UPPER(airline_name) AS uppercase_name
FROM airline
ORDER BY airline_id;
```

RESULT TABLE | 6 rows

uppercase_name
AIR FRANCE
AIR PORTUGAL
POLISH AIR
KAZAIR
GLOBAL AIRWAYS
AIREASY

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 2

Replace any occurrence of "Air" in airline names with "Aero".

EXPLANATION

REPLACE performs a case-sensitive substring replacement, including Air inside KazAir and Airways. The stored names are unchanged.

VERIFIED RESULT

6 rows, including Aero France, KazAero and Global Aeroways.

TASK 02 | SQL AND VERIFIED RESULT

```
SELECT REPLACE(airline_name, 'Air', 'Aero') AS new_name
FROM airline
ORDER BY airline_id;
```

RESULT TABLE | 6 rows

new_name
Aero France
Aero Portugal
Polish Aero
KazAero
Global Aeroways
AeroEasy

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 3

Find flight numbers associated with airline 1 or airline 2.

EXPLANATION

The assignment says "both", but each flight has one airline_id and flight_id is unique. IN (1, 2) lists flights from either airline, covering both groups. A flight belonging to both would require a different schema. Here flight_id is the available flight identifier; no flight_number column exists.

VERIFIED RESULT

4 rows: flight IDs 101, 102, 103 and 104.

TASK 03 | SQL AND VERIFIED RESULT

```
SELECT flight_id, airline_id
FROM flights
WHERE airline_id IN (1, 2)
ORDER BY flight_id;
```

RESULT TABLE | 4 rows

flight_id	airline_id
101	1
102	2
103	1
104	2

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 4

Retrieve airports containing both "Reginal" and "Air" in their names.

EXPLANATION

Both case-sensitive substring conditions must match. The spelling Reginal is preserved exactly as supplied in the assignment.

VERIFIED RESULT

2 rows: airports 1 and 2.

TASK 04 | SQL AND VERIFIED RESULT

```
SELECT airport_id, airport_name
FROM airport
WHERE airport_name LIKE '%Reginal%'
  AND airport_name LIKE '%Air%'
ORDER BY airport_id;
```

RESULT TABLE | 2 rows

airport_id	airport_name
1	Paris Reginal Air Hub
2	Lisbon Reginal Air Terminal

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 5

Retrieve passenger names and birth dates formatted as Month DD, YYYY.

EXPLANATION

TO_CHAR formats the date as text. FM removes padding after the month name, producing an output such as April 12, 1990. The underlying date stays unchanged.

VERIFIED RESULT

5 rows; Alice Martin has April 12, 1990.

TASK 05 | SQL AND VERIFIED RESULT

```
SELECT first_name, last_name,
       TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS birth_date
FROM passengers
ORDER BY passenger_id;
```

RESULT TABLE | 5 rows

first_name	last_name	birth_date
Alice	Martin	April 12, 1990
Bruno	Silva	September 23, 1985
Clara	Nowak	January 05, 2000
Daniel	Brown	December 30, 1995
Emma	Wilson	August 19, 2005

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 6

Find flight numbers delayed based on actual arrival time.

EXPLANATION

A flight is delayed when its actual arrival is later than its scheduled arrival. flight_id serves as the flight identifier in this schema.

VERIFIED RESULT

3 rows: flight IDs 101, 103 and 105.

TASK 06 | SQL AND VERIFIED RESULT

```
SELECT flight_id
FROM flights
WHERE act_arrival_time > sch_arrival_time
ORDER BY flight_id;
```

RESULT TABLE | 3 rows

flight_id
101
103
105

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 7

Find flights that arrived later than scheduled.

EXPLANATION

This task uses the same condition as Task 6. Showing both timestamps makes each late arrival easy to verify.

VERIFIED RESULT

3 rows: 101 (30 minutes late), 103 (45 minutes), 105 (20 minutes).

TASK 07 | SQL AND VERIFIED RESULT

```
SELECT flight_id, sch_arrival_time, act_arrival_time
FROM flights
WHERE act_arrival_time > sch_arrival_time
ORDER BY flight_id;
```

RESULT TABLE | 3 rows

flight_id	sch_arrival_time	act_arrival_time
101	2024-06-01 10:00:00	2024-06-01 10:30:00
103	2024-06-03 12:00:00	2024-06-03 12:45:00
105	2024-06-05 14:00:00	2024-06-05 14:20:00

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 8

Show airlines from France, Portugal or Poland created from November 1, 2023 through March 31, 2024.

EXPLANATION

IN accepts any of the three countries. The exclusive April 1 upper bound includes every time on March 31. A midnight March 31 upper bound would omit records created later that day.

VERIFIED RESULT

3 rows: airlines 1, 2 and 3; airline 2 is included at March 31, 18:30.

TASK 08 | SQL AND VERIFIED RESULT

```
SELECT airline_id, airline_name, airline_country, created_at
FROM airline
WHERE airline_country IN ('France', 'Portugal', 'Poland')
  AND created_at >= TIMESTAMP '2023-11-01 00:00:00'
  AND created_at <  TIMESTAMP '2024-04-01 00:00:00'
ORDER BY airline_id;
```

RESULT TABLE | 3 rows

airline_id	airline_name	airline_country	created_at
1	Air France	France	2023-11-01 00:00:00
2	Air Portugal	Portugal	2024-03-31 18:30:00
3	Polish Air	Poland	2024-01-15 09:00:00

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 9

Find the top 3 heaviest baggage items weighing more than 25 kg.

EXPLANATION

Filter overweight items, sort from heaviest to lightest, then keep three. baggage_id gives a consistent order if weights are equal.

VERIFIED RESULT

3 rows: baggage 1 (32.50 kg), 2 (29.00 kg) and 3 (27.50 kg).

TASK 09 | SQL AND VERIFIED RESULT

```
SELECT baggage_id, weight_in_kg
FROM baggage
WHERE weight_in_kg > 25
ORDER BY weight_in_kg DESC, baggage_id
LIMIT 3;
```

RESULT TABLE | 3 rows

baggage_id	weight_in_kg
1	32.50
2	29.00
3	27.50

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 10

Find the youngest passenger's full name.

EXPLANATION

The most recent birth date identifies the youngest passenger. || joins the first and last names with a space. If birth dates tie, the lowest passenger ID is selected.

VERIFIED RESULT

1 row: Emma Wilson (born August 19, 2005).

TASK 10 | SQL AND VERIFIED RESULT

```
SELECT first_name || ' ' || last_name AS full_name
FROM passengers
ORDER BY date_of_birth DESC, passenger_id
LIMIT 1;
```

RESULT TABLE | 1 row

full_name
Emma Wilson

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 11

Find the cheapest booking price on each booking platform.

EXPLANATION

GROUP BY forms one group per platform. MIN returns the lowest ticket price within each group.

VERIFIED RESULT

3 rows: Mobile App 95.00, Travel Agency 175.00, Website 120.00.

TASK 11 | SQL AND VERIFIED RESULT

```
SELECT booking_platform, MIN(ticket_price) AS cheapest_price
FROM booking
GROUP BY booking_platform
ORDER BY booking_platform;
```

RESULT TABLE | 3 rows

booking_platform	cheapest_price
Mobile App	95.00
Travel Agency	175.00
Website	120.00

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 12

Return airlines whose airline_code contains a digit.

EXPLANATION

The PostgreSQL ~ operator checks a regular expression. [0-9] matches any digit anywhere in the code.

VERIFIED RESULT

4 rows: airlines 1 (AF1), 2 (TP2), 4 (KZ7) and 6 (FR6).

TASK 12 | SQL AND VERIFIED RESULT

```
SELECT airline_id, airline_code, airline_name
FROM airline
WHERE airline_code ~ '[0-9]'
ORDER BY airline_id;
```

RESULT TABLE | 4 rows

airline_id	airline_code	airline_name
1	AF1	Air France
2	TP2	Air Portugal
4	KZ7	KazAir
6	FR6	AirEasy

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 13

List the top 5 most recently created airlines.

EXPLANATION

DESC puts the newest creation timestamps first. LIMIT 5 returns five airlines; airline_id breaks timestamp ties consistently.

VERIFIED RESULT

5 rows, in order: airline IDs 5, 4, 2, 3 and 1.

TASK 13 | SQL AND VERIFIED RESULT

```
SELECT airline_id, airline_name, created_at
FROM airline
ORDER BY created_at DESC, airline_id
LIMIT 5;
```

RESULT TABLE | 5 rows

airline_id	airline_name	created_at
5	Global Airways	2024-05-10 09:00:00
4	KazAir	2024-04-05 09:00:00
2	Air Portugal	2024-03-31 18:30:00
3	Polish Air	2024-01-15 09:00:00
1	Air France	2023-11-01 00:00:00

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 14

Return baggage-check rows with booking_id from 200 to 300 inclusive and check_result other than Checked.

EXPLANATION

BETWEEN includes both endpoints, 200 and 300. <> means not equal. SELECT * returns all columns of each matching baggage-check row.

VERIFIED RESULT

3 rows: check IDs 2, 3 and 5, for bookings 200, 220 and 300.

TASK 14 | SQL AND VERIFIED RESULT

```
SELECT *
FROM baggage_check
WHERE booking_id BETWEEN 200 AND 300
  AND check_result <> 'Checked'
ORDER BY baggage_check_id;
```

RESULT TABLE | 3 rows

Record details

baggage_check_id	check_result	booking_id	passenger_id
2	Not checked	200	2
3	Flagged	220	3
5	Not checked	300	5

Timestamps (same records)

baggage_check_id	created_at	updated_at
2	2024-06-02 08:00:00	2024-06-01 08:00:00
3	2024-06-03 09:00:00	2024-06-03 08:30:00
5	2024-06-05 11:00:00	2024-05-31 11:00:00

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.

Task 15

Find baggage checks updated earlier than they were created, within the same calendar month.

EXPLANATION

DATE_TRUNC compares the calendar month and year together. The second condition finds an earlier update. Such timestamps are deliberate data-quality anomalies in this sample. The schema column is updated_at, correcting the assignment's update_at typo.

VERIFIED RESULT

2 rows: check IDs 2 and 3. Checks 5 (different month) and 6 (different year) are excluded.

TASK 15 | SQL AND VERIFIED RESULT

```
SELECT *
FROM baggage_check
WHERE DATE_TRUNC('month', updated_at)
      = DATE_TRUNC('month', created_at)
  AND updated_at < created_at
ORDER BY baggage_check_id;
```

RESULT TABLE | 2 rows

Record details

baggage_check_id	check_result	booking_id	passenger_id
2	Not checked	200	2
3	Flagged	220	3

Timestamps (same records)

baggage_check_id	created_at	updated_at
2	2024-06-02 08:00:00	2024-06-01 08:00:00
3	2024-06-03 09:00:00	2024-06-03 08:30:00

Source: verified PostgreSQL 18.3 output. Generated illustration; not a pgAdmin screenshot.